LEVEL 2 OF 3 · MEDIUM

Architecture Guide 2: A departmental AI platform

One organisation, several departments, real users and real permissions. How to grow from a single assistant to a service with tools, roles, evaluation gates, monitoring and a release process.

USERS 500 to 5,000	**TEAM** 4 to 8 engineers	**BUILD TIME** 6 to 10 weeks
PATTERN RAG + tools + agent loop	**RUNS ON** Managed containers	**ENVIRONMENTS** Dev, test, production

Nagarjuna Naidu | AI Solution Architect program | October 2026

1. What changes from Level 1

Level 1 answered questions from one set of documents. The **Northwind Service Assistant** in this guide serves HR, Finance, Travel and IT. People ask questions that need **documents** (what is the policy), **data** (how many bookings were over the cap) and **calculation** (what is the total overage), and different people may see different things.

Level 1	Level 2	Why it changes
One service	API, agent, tools and ingestion as separate parts	Different load, different release speed, different owners
One audience	Roles and permissions on every request and every document	Some content is confidential
Prompt plus retrieval	An agent that chooses between search, SQL and calculator	Questions need more than documents
Manual test set	Automated evaluation that blocks a bad release	Many people change things; regressions are certain
Log file	Tracing, dashboards, drift checks and feedback loop	You must know it is getting worse before users tell you
Deploy by hand	Pipeline: test, build, deploy, roll back	Repeatable and safe releases

2. The architecture

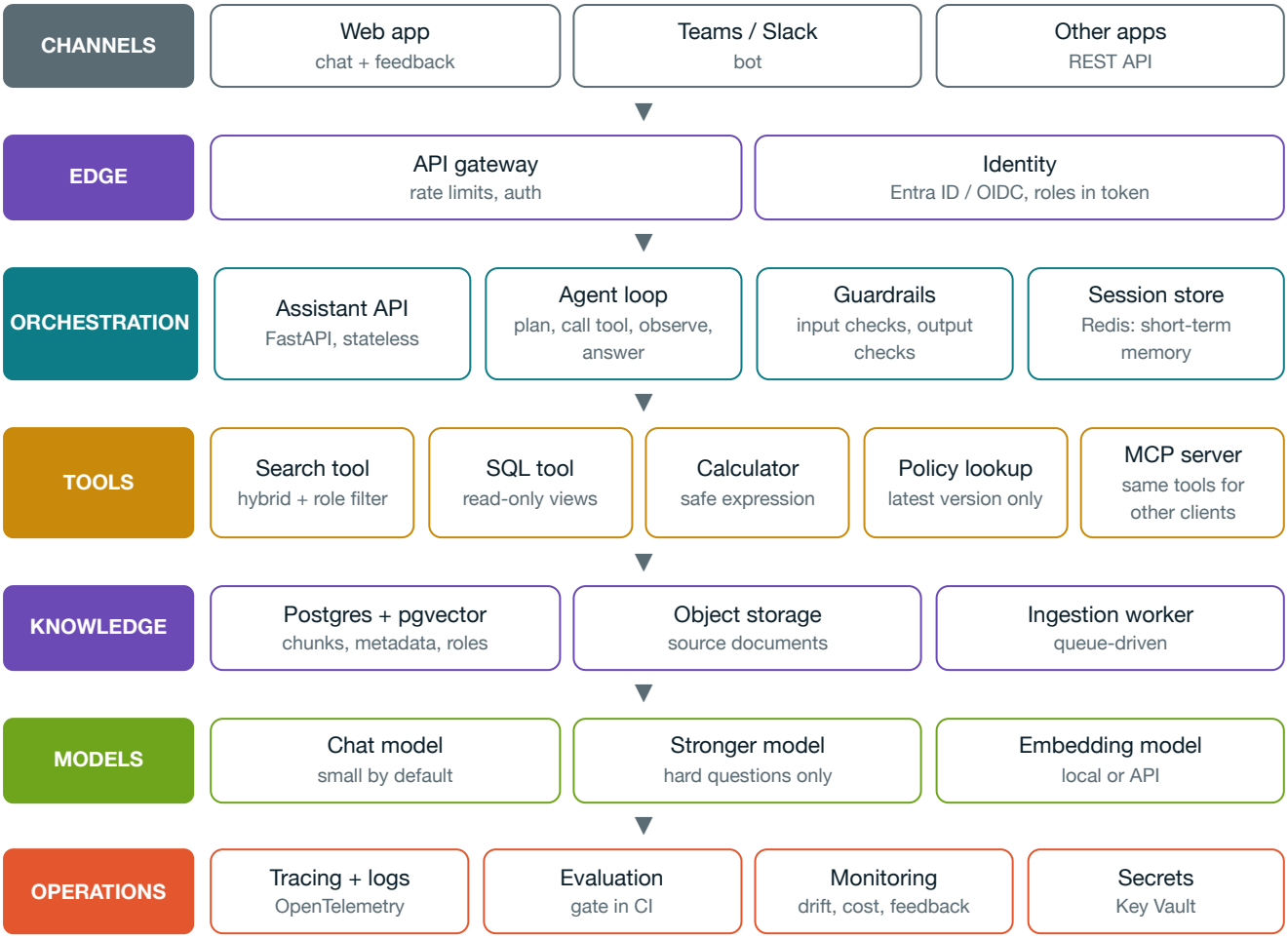


Figure 1. Level 2 layers. Services are stateless; state lives in Redis, Postgres and storage.

Request flow with roles and tools

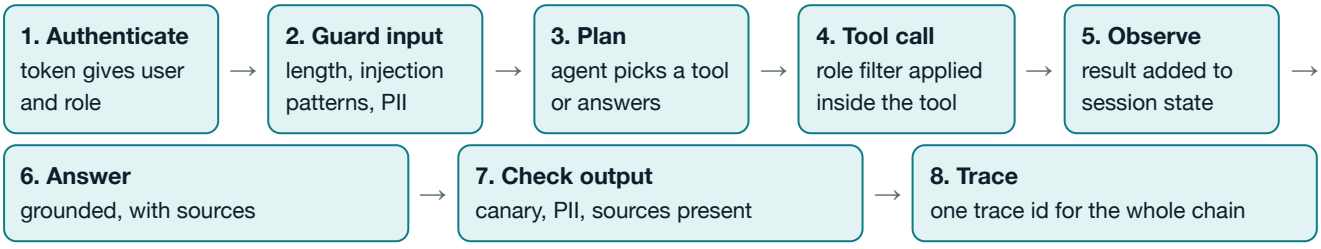


Figure 2. Every step is traced under one id, so you can replay a bad answer.

3. Components and the choices behind them

Component	Its job	Default choice	Notes
API gateway	One front door: authentication, quotas, throttling	Azure API Management or Kong	Gives you per-team rate limits and an audit log without code

Identity	Who is asking, which role	Entra ID (OIDC) with a role claim	Never trust a role sent in the request body
Assistant API	Receives the question, runs the agent, returns the answer	FastAPI in a container	Stateless, so you can run 2 or more copies
Agent loop	Choose and sequence tools, limited steps	Small hand-written loop (about 100 lines) or LangGraph	Start simple; adopt a framework when you need persistence or branching
Session store	Short-term memory per conversation	Redis with an expiry	Keep the last few turns plus a summary
Retrieval	Find relevant chunks the user may see	Hybrid (vector + keyword) in pgvector / Azure AI Search	Apply the role filter in the query, not after
Tools	Do one thing each with validated inputs	Python functions in a registry	Allow-list per role; read-only by default
MCP server	Expose the same tools to any compatible client	MCP Python SDK (FastMCP)	One registry feeds both the agent and the MCP server
Models	Answer generation and embeddings	Small chat model by default, stronger on escalation	Route by difficulty to control cost
Evaluation	Prove a change did not make things worse	Golden set + automated scoring in CI	Fail the build below the threshold
Monitoring	See quality, cost, drift and use	Tracing + dashboard + weekly drift report	Population stability (PSI) on query features
Infrastructure as code	Recreate environments reliably	Terraform or Bicep	No manual clicking in production

4. Complete folder structure

```
northwind-service-assistant/
├── README.md
├── Makefile                      make test | make eval | make run | make deploy-dev
├── pyproject.toml                one dependency list, pinned
├── .env.example
├── .github/
│   └── workflows/
│       ├── ci.yml                lint, unit tests, evaluation gate
│       └── deploy.yml            build image, push, deploy dev -> test -> prod
├── infra/                        infrastructure as code
│   ├── main.tf                   resource group, container apps, redis, postgres
│   ├── variables.tf
│   └── envs/
│       ├── dev.tfvars
│       ├── test.tfvars
│       └── prod.tfvars
├── services/
│   ├── api/                      the public service
│   │   ├── Dockerfile
│   │   ├── main.py               routes: /ask /feedback /health
│   │   ├── auth.py               verify token, extract role
│   │   ├── guardrails.py         input + output checks
│   │   └── session.py            Redis helpers
│   ├── agent/
│   │   ├── loop.py               plan -> act -> observe, step limit
│   │   ├── planner.py            LLM planner + offline fallback rules
│   │   └── prompts/
│   │       ├── system.md
│   │       └── planner.md
│   ├── tools/
│   │   ├── registry.py           name -> function, allowed roles
│   │   ├── search_docs.py
│   │   ├── query_sql.py          read-only, one SELECT, row limit
│   │   ├── calculator.py
│   │   └── lookup_policy.py
│   ├── mcp_server/
│   │   └── server.py             exposes the registry over MCP
│   └── ingestion/
│       ├── worker.py             listens to a queue, indexes new files
│       ├── chunking.py
│       └── metadata.py           sensitivity, version, owner
├── db/
│   ├── schema.sql                chunks, documents, feedback, query_log
│   └── views.sql                 safe read-only views for the SQL tool
├── eval/
│   ├── golden_set.json           200 questions with expected source and answer
│   ├── attacks.json              injection and leakage tests
│   ├── run_eval.py               scores and writes results to MLflow
│   └── gate.py                   exit 1 if below thresholds
├── monitoring/
│   ├── drift.py                  PSI on query features
│   ├── dashboard.json            cost, latency, refusals, feedback
│   └── alerts.yaml
├── tests/
│   ├── unit/
│   └── integration/
└── docs/
    ├── architecture.md           diagrams and decisions
```

```
    ├── runbook.md              what to do when it breaks
    └── data_catalog.md         which documents, who owns them, sensitivity
```

5. The key ideas in code

Role filter inside the retrieval query (never after)

```
ROLE_LEVELS = {"intern": ["public"], "employee": ["public", "internal"],
               "hr_partner": ["public", "internal", "confidential"]}

def search_docs(query_vec, role, k=3):
    allowed = ROLE_LEVELS[role]
    rows = db.execute("""
        SELECT title, text, 1 - (embedding <=> %s) AS score
        FROM chunks
        WHERE sensitivity = ANY(%s) AND is_latest = true
        ORDER BY embedding <=> %s LIMIT %s""", (query_vec, allowed, query_vec, k))
    return [r for r in rows if r["score"] >= MIN_SIM]
```

Tool registry with an allow-list per role

```
TOOLS = {
    "search_docs":   {"fn": search_docs,   "roles": {"intern", "employee", "hr_partner"}},
    "query_sql":     {"fn": query_sql,     "roles": {"employee", "hr_partner"}},
    "calculator":    {"fn": calculator,    "roles": {"intern", "employee", "hr_partner"}},
    "lookup_policy": {"fn": lookup_policy, "roles": {"employee", "hr_partner"}},
}

def call_tool(name, args, role):
    spec = TOOLS.get(name)
    if spec is None or role not in spec["roles"]:
        return {"error": f"tool {name} is not available"}      # same message for unknown and
forbidden
    try:
        return {"result": spec["fn"](**args)}
    except Exception as e:                                     # never crash the loop on a tool
error
        return {"error": f"{type(e).__name__}: {e}"}
```

The agent loop: small, limited, observable

```
def run(question, role, session, max_steps=5):
    scratch = []
    for step in range(max_steps):
        decision = plan(question, scratch, allowed=tools_for(role))   # LLM, with offline rules
as fallback
        trace("plan", step=step, **decision)
        if decision["action"] == "finish":
            break
        obs = call_tool(decision["action"], decision["input"], role)
        trace("tool", step=step, tool=decision["action"], ok="error" not in obs)
        scratch.append((decision, obs))
    return final_answer(question, scratch, session)                   # always cites sources, may
refuse
```

Expose the same tools over MCP

```
from mcp.server.fastmcp import FastMCP
from services.tools.registry import TOOLS

mcp = FastMCP("northwind")

@mcp.tool()
def search_docs(query: str, k: int = 3) -> str:
    """Search company documents (read-only)."""
    return TOOLS["search_docs"]["fn"](query, k=k)

if __name__ == "__main__":
    mcp.run()          # stdio for local clients; use an HTTP transport behind the gateway in
production
```

The evaluation gate in the pipeline

```
# .github/workflows/ci.yml (excerpt)
- name: Unit tests
  run: pytest tests/unit -q
- name: Evaluation gate
  run: python eval/run_eval.py && python eval/gate.py --min-correct 0.85 --min-refusal 0.95 --
max-leaks 0
  env:
    AZURE_OPENAI_ENDPOINT: ${{ secrets.AZURE_OPENAI_ENDPOINT }}
    AZURE_OPENAI_API_KEY:  ${{ secrets.AZURE_OPENAI_API_KEY }}
```

6. Step-by-step build plan

Phase	Weeks	What you build	Exit criteria
1. Foundations	1	Repo, environments, secrets, CI skeleton, Terraform for dev, data catalog (documents, owners, sensitivity)	Hello-world API deploys to dev from the pipeline
2. Knowledge	2	Ingestion worker, chunking, metadata (sensitivity, version), pgvector schema, role-filtered search	Golden set retrieval hit rate 90%+, roles verified by tests
3. Tools and agent	2	Tool registry, read-only SQL views, calculator, agent loop, offline fallback, step limit	Multi-step questions answered with sources; tool errors do not crash the loop
4. Safety	1	Input and output guardrails, injection tests, PII masking, canary check, refusal rules	Attack set: zero leaks; refusals correct
5. Evaluation and monitoring	1	Golden set of 200, MLflow tracking, CI gate, tracing, dashboard, drift report, feedback endpoint	Gate blocks a deliberately bad change
6. Release	1 to 2	Test and prod environments, load test, runbook, pilot with one department, then rollout	Pilot signed off by the data owner

7. Security, governance and compliance

Area	What to put in place
Identity and roles	Role comes from the signed token; enforced in the tool and in the query; a test per role per tool
Data classification	Every document has an owner and a sensitivity level; confidential content is only indexed with the right label
Prompt injection	Treat retrieved text as data; scan chunks at ingestion and at answer time; test with a standing attack set
Privacy	Mask personal data in logs; retention period for logs and feedback; a way to delete a document and its chunks
Tool safety	Read-only by default; one statement, row limit, blocked keywords for SQL; write tools only with human approval
Auditability	Trace id per request; who asked, which role, which tools, which sources, which model and prompt version
Model governance	Prompt and model versions in git; change requires a passing evaluation gate and an approver

8. Evaluation, monitoring and cost control

Signal	Measured by	Alert when
Correctness and grounding	Nightly run of the golden set	Drop of 5 points or more
Refusal quality	Out-of-scope questions in the golden set	Any wrong answer where a refusal was expected
Security	Attack set on every release	Any leak
Latency	Trace durations, median and slowest 5%	Slowest 5% above 10 s
Cost per question	Tokens by model and tool	Rising trend week on week
Input drift	PSI on question length, topic mix, role mix	PSI above 0.25 (investigate at 0.1)
User feedback	Thumbs up and down with comment	Thumbs-down rate above baseline

Cost levers, in order of effect

1) small model by default and a stronger one only when the first answer is low-confidence, 2) short context: three good chunks beat ten mediocre ones, 3) cache repeated questions (for example common policy questions) for a short time, 4) limit the agent to five steps, 5) summarise old turns in the session instead of resending them.

9. Deployment

- **Environments:** dev (anyone), test (release candidates, real-like data), prod (approved only). Same Terraform, different variable files.
- **Runtime:** managed containers with 2 or more replicas of the API; ingestion worker scaled by queue length.
- **Release:** pull request, CI (tests and evaluation gate), merge, deploy to dev and test automatically, production after approval.
- **Rollback:** keep the previous image and previous prompt version; roll back in one command.
- **Rate limits and quotas:** set per team at the gateway, and a monthly budget alert on the model resource.

10. Common mistakes at this level

- Filtering by role **after** retrieval, so restricted text reaches the model.
- One giant tool that does everything, instead of small tools with validated inputs.
- No step limit and no timeout, so one slow tool stalls the whole request.
- Changing the prompt or model without rerunning the evaluation set.
- Logging full questions with personal data and keeping them forever.
- Letting a framework hide the control flow before you understand it.

11. Checklist before production

- □ Role tests pass for every tool and every role
- □ Evaluation gate in CI, thresholds agreed with the business owner
- □ Attack set passes with zero leaks
- □ Dashboard and alerts live; someone on call
- □ Runbook written and tried once
- □ Data owners signed off on the documents indexed
- □ Budget alert and rate limits set

12. When this level stops being enough

> **Signals to move to Level 3**
> Several business units each want their own assistants and data; workflows that take actions and need approvals; regulators or auditors ask for evidence; more than one region or cloud; hundreds of thousands of users; or teams keep rebuilding the same plumbing. Level 3 turns the assistant into a **platform** that other teams build on, with a shared gateway, governed model access, an agent runtime, long-term memory and central controls.